

C++ OOP Cheat Sheet

Core syntax for every OOP pillar — read top to bottom to build understanding.

C++

01 Class & Object

Blueprint + instance — the base unit of OOP.

```
class ClassName {
public:
    int data;           class="cm">// member variable
    void show() { }     class="cm">// member function
};

int main() {
    ClassName obj;      class="cm">// create object
    obj.data = 10;
    obj.show();
}
```

02 Encapsulation

Bind data + methods together; hide internals.

```
class Car {
private:
    string brand;        class="cm">// hidden data

public:
    void setBrand(string b) {    class="cm">// setter
        brand = b;
    }
    string getBrand() {        class="cm">// getter
        return brand;
    }
};
```

03 Abstraction

Show what an object does, hide how.

```
class Shape {
public:
    virtual void draw() = 0;    class="cm">// pure virtual -> abstract class
};

class Circle : public Shape {
public:
    void draw() override {     class="cm">// must implement
        class="cm">// drawing logic
    }
};
```

04 Inheritance

A class acquires properties of another class.

```
class Animal {
public:
    void eat() { }
};

class Dog : public Animal {    class="cm">// single inheritance
public:
    void bark() { }
};

class A { };
class B { };
class C : public A, public B { };    class="cm">// multiple inheritance
```

05 Polymorphism

One interface, many forms — compile & run time.

```
class="cm">// Compile-time: function overloading
class MathOp {
public:
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
};

class="cm">// Run-time: virtual + overriding
class Base {
public:
    virtual void show() { }
};

class Derived : public Base {
public:
    void show() override { }
};
```

```
Base* b = new Derived();
b->show();           class="cm">// resolved at run time
```

06 Constructor

Special function that runs when an object is created.

```
class Point {
public:
    int x, y;

    Point() {           class="cm">// default constructor
        x = 0; y = 0;
    }
    Point(int a, int b) {   class="cm">// parameterized
        x = a; y = b;
    }
    Point(const Point &p) {   class="cm">// copy constructor
        x = p.x; y = p.y;
    }
};

Point p1;           class="cm">// default
Point p2(3, 4);     class="cm">// parameterized
Point p3 = p2;      class="cm">// copy
```

07 Destructor

Special function that runs when an object is destroyed.

```
class Point {
public:
    ~Point() {           class="cm">// destructor: ~ClassName()
        class="cm">// cleanup / release resources
    }
};

int main() {
    Point p;           class="cm">// constructed
    class="cm">// destructor called here (out of scope)
}
```

08 Access Modifiers

Control visibility of class members.

```
class Demo {
public:
    int a;           class="cm">// accessible from anywhere

protected:
    int b;           class="cm">// accessible in class + derived classes

private:
    int c;           class="cm">// accessible only inside this class
};
```

09 Static Keyword

Belongs to the class, shared by all objects.

```
class Counter {
public:
    static int count;     class="cm">// declaration (shared by all objects)

    Counter() {
        count++;
    }
};

int Counter::count = 0;   class="cm">// definition – outside the class

static void utilityFn() { } class="cm">// static function: internal linkage
```

10 this Pointer

Pointer to the current object inside a member function.

```
class Point {
public:
    int x;
    Point(int x) {
        this->x = x;       class="cm">// disambiguates member vs parameter
    }

    Point& setX(int x) {
        this->x = x;
        return *this;     class="cm">// enables method chaining
    }
};
```